ML Project Report

Team 12
Course: CMP4040

Name	ID
Kirollos Baghdad	9210859
Ali Sami	9220497
Ahmed El-Sayed Mohsen	9220019
Rawan Ahmed	9220302

Idea: Malicious Network Flow Classification

1. Problem Definition

The task is to classify network flows as either **benign** or **malicious** using tabular connection-level features from **IoT-23**. This is a **supervised binary classification problem** on network traffic logs.

2. Motivation

Malicious traffic detection is useful for **cybersecurity** because intrusion detection systems must catch suspicious behavior while keeping **false alarms low**.

3. Evaluation Metrics: accuracy, precision, recall, F1-score, confusion matrix, ROC-AUC

4. Dataset Used

- Malware Detection in Network Traffic Data
  - It is **tabular connection-level data**
  - It is related to labeled malware/benign traffic from the **IoT-23 / Stratosphere Laboratory** line of datasets

5. Research Papers

- A Comparative Analysis of Machine Learning Techniques for IoT Intrusion Detection
- Evaluation and Selection Models for Ensemble Intrusion Detection Systems in IoT

6. Dataset Analysis Results

- The latest notebook run used 14 capture files and produced the following raw dataset summary:

Item	Value
Raw feature count	22
Raw sample count	329,344
File Count	14

- **Raw class distribution:**

Class	Rows
malicious	203,284
benign	126,060

- **Main missing columns**

Column	Missing values
local_resp	329,344
local_orig	329,344
tunnel_parents	329,344
service	317,634
detailed_label	307,093
orig_bytes	220,919
resp_bytes	220,919
duration	220,919

These numbers show two important issues.
First, the raw dataset is imbalanced. Second, several features are heavily missing, so cleaning and encoding are necessary before training.

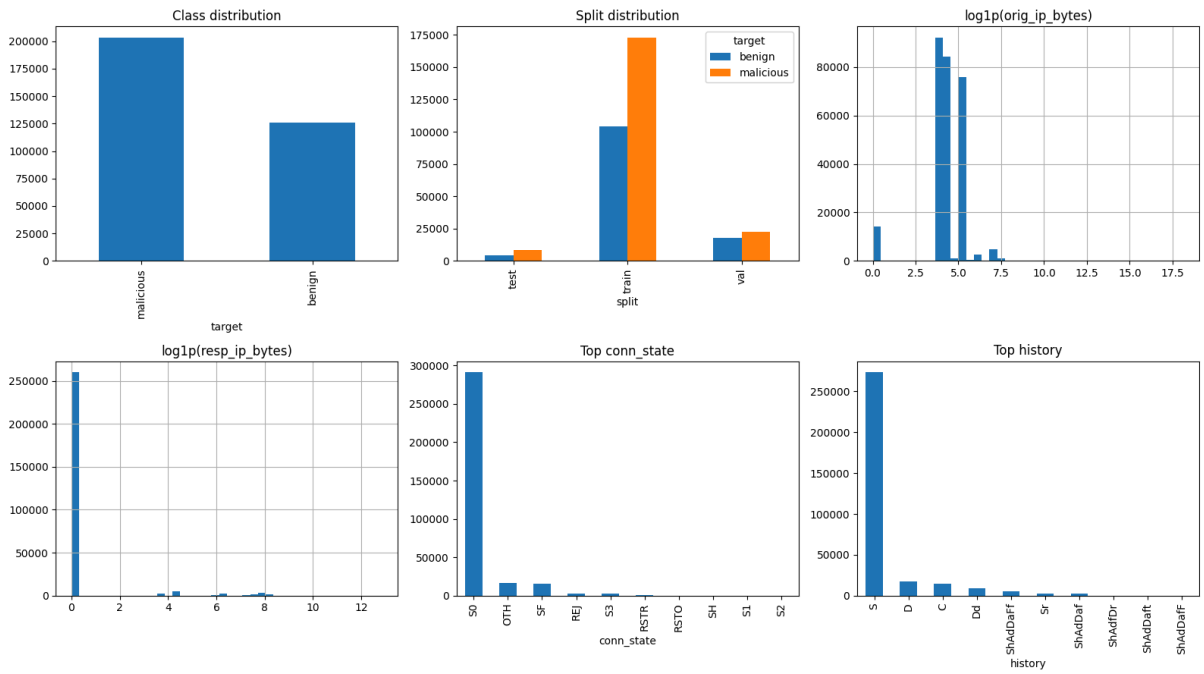

Figure 1. Dataset analysis overview: class distribution, split distribution, feature histograms, and top categorical patterns.

7. Experimental Setup

- **Data Splitting**

The workflow uses a capture-based split. This is stricter than a row-based split because validation and test flows come from unseen captures.

Split	Rows	Label distribution
train	208,144	benign: 104,072, malicious: 104,072
val	40,000	malicious: 22,251, benign: 17,749
test	12,461	malicious: 8,222, benign: 4,239

Only the training split is balanced. Validation and test keep their natural capture-level class mix.

8. Preprocessing

Input features used for modeling:

- **Numeric**: id.resp_p, duration, orig_bytes, resp_bytes, missed_bytes, orig_pkts, orig_ip_bytes, resp_pkts, resp_ip_bytes
- **Categorical**: proto, service, conn_state, history
- **Boolean columns when present**: local_orig, local_resp

Main preprocessing steps:

- **Replace - and (empty)** with missing values
- Convert usable boolean values from **T/F to 1/0**
- **Median imputation** for numeric features
- **Most-frequent imputation** for categorical features
- **Standard scaling** for numeric features
- **One-hot encoding** for categorical features

Split	Rows	Input features	Output features
train	208,144	13	126
val	40,000	13	126
test	12,461	13	126

9. Experimental Results

- **Baseline**

ZeroR predicts the majority class from the training data.

Model	Predicted Class	Accuracy	Precision	Recall	F1
ZeroR	malicious	0.556275	0.556275	1.00	0.714880

This gives a useful lower bound. It achieves perfect recall on validation only because it predicts every flow as malicious, which is not good enough on its own.

10. Model Settings

1. **Logistic Regression**: C=1.0, penalty=l2, solver=lbfgs, max_iter=1000
2. **Linear SVM:** C=1.0, loss=squared_hinge, dual=False, max_iter=10000
3. **Random Forest**: n_estimators=300, max_depth=2, min_samples_split=8000, min_samples_leaf=3000, max_leaf_nodes=6, max_features=0.3, class_weight=balanced_subsample, max_samples=0.25, ccp_alpha=0.01, n_jobs=-1

11. Validation Results

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.975000	0.972265	0.983102	0.977654
Linear SVM	0.975650	0.973390	0.983102	0.978222
Random Forest	0.936450	0.901913	0.993843	0.945649
ZeroR	0.556275	0.556275	1.000000	0.714880

- **Training versus validation F1:**

Model	Train F1	Val F1
Logistic Regression	0.922513	0.977654
Linear SVM	0.922569	0.978222
Random Forest	0.926488	0.945649

Linear SVM has the best validation F1, with Logistic Regression very close.

12. Final Test

The final selected model is Linear SVM.

Model	Accuracy	Precision	Recall	F1
Linear SVM	0.966616	0.951841	1.00	0.975326

- **Confusion matrix**

Actual \ Predicted	benign	malicious
benign	3,823	416
malicious	0	8,222

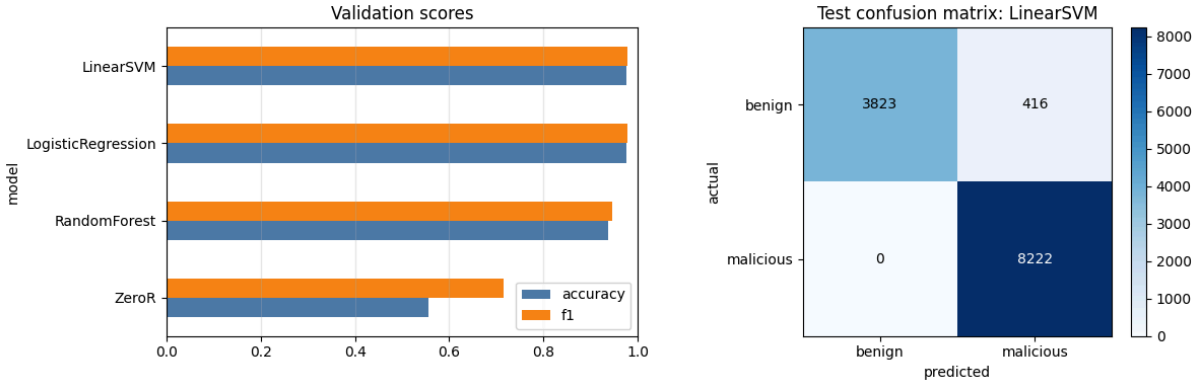

Figure 2. Validation comparison and final test confusion matrix.

Test ROC-AUC: 0.996226

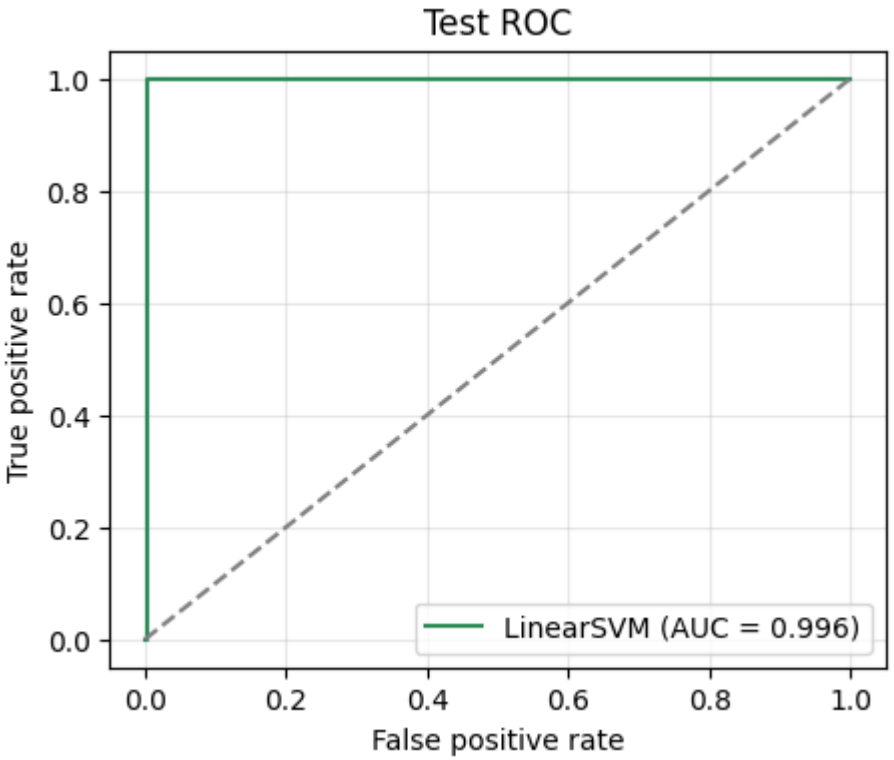

Figure 3. ROC curve of the final test result

13. Feature Analysis

The final model is Linear SVM, so feature contributions are based on the learned coefficients after preprocessing.

Top original input features:

Rank	Input feature	Importance
1	history	25.138241
2	orig_ip_bytes	12.528733
3	conn_state	4.855729
4	service	4.339653

The strongest features are mostly behavior-based features such as history, conn_state, service, and orig_ip_bytes. This suggests that both connection behavior and traffic size patterns matter for malicious flow detection.

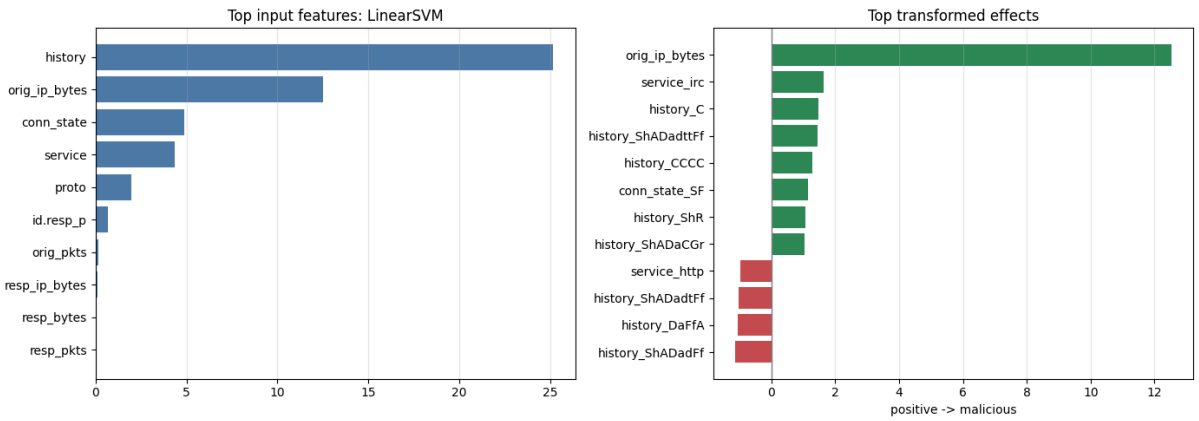

Figure 4. Feature contribution for the final model

14. Discussion

The results show that the learned models clearly outperform the **ZeroR baseline**.

Logistic Regression and Linear SVM are the strongest and most stable models in this task.
Random Forest performs lower than the linear models, but it is strongly regularized.

The final **Linear SVM test result is strong, with an F1 Score of 0.975326 and an ROC AUC of 0.996226**. The model misses no malicious flows in this test run, but it still predicts **416 benign** flows as **malicious**, so false alarms are still present.

The **feature analysis** also supports the result. The model depends mostly on **behavior-oriented features**, which is reasonable for **malicious flow detection**.
The evaluation is **capture-based**, which is better than **row-based splitting**.

15. Conclusion

This project built a **full machine learning pipeline** for **binary malicious-flow classification** on **IoT-23 v2**. It included **dataset analysis, preprocessing, baseline evaluation**, **training of three non-deep-learning models, model comparison, final testing, and feature analysis**.

After experimenting, **Linear SVM** was chosen as the **final model**.
It achieved the **best validation F1** and generalized well on the test set.

Thank You